

Week8 Ways to gate

Macarena López-Mayorga

2026-06-11

First, we have to check if we have all the necessary packages installed and the proper versions:

```
packageVersion("ggplot2")
```

```
[1] '4.0.3'
```

```
packageVersion("ggcyto")
```

```
[1] '1.39.4'
```

```
packageVersion("openCyto")
```

```
[1] '2.22.0'
```

```
packageVersion("flowGate")
```

```
[1] '1.10.0'
```

This version of flowGate seems to have problems with shiny app because there are some bugs not resolved. David Rach and his colleagues have developed a forked version that fix the bug problems. So, I am going to remove flowGate package and install the version of David Rach and others:

```
remove.packages("flowGate")
```

Now, I have to call the remotes packages to install from the github repository

```
library(remotes)
```

And install the forked version:

```
install_github("DavidRach/flowGate")
```

Let's check now if we have the correct version:

```
packageVersion("flowGate")
```

```
[1] '0.99.2'
```

Yes! that is correct :) . Now, we have to load in several packages that will be necessary:

```
library(flowWorkspace)
library(ggplot2)
library(ggcyto)
library(flowGate)
library(openCyto)
```

```
StorageLocation <- file.path("C:/Users/Macarena/CytometryInR/course/08_WaysToGate/data")
fcs_files <- list.files(StorageLocation, pattern=".fcs", full.names=TRUE)
fcs_files
```

```
[1] "C:/Users/Macarena/CytometryInR/course/08_WaysToGate/data/2025_07_26_AB_02_INF052_00_Ct"
[2] "C:/Users/Macarena/CytometryInR/course/08_WaysToGate/data/2025_07_26_AB_02_INF052_00_SE"
[3] "C:/Users/Macarena/CytometryInR/course/08_WaysToGate/data/2025_07_26_AB_02_INF100_00_Ct"
[4] "C:/Users/Macarena/CytometryInR/course/08_WaysToGate/data/2025_07_26_AB_02_INF100_00_SE"
[5] "C:/Users/Macarena/CytometryInR/course/08_WaysToGate/data/2025_07_26_AB_02_INF179_00_Ct"
[6] "C:/Users/Macarena/CytometryInR/course/08_WaysToGate/data/2025_07_26_AB_02_INF179_00_SE"
```

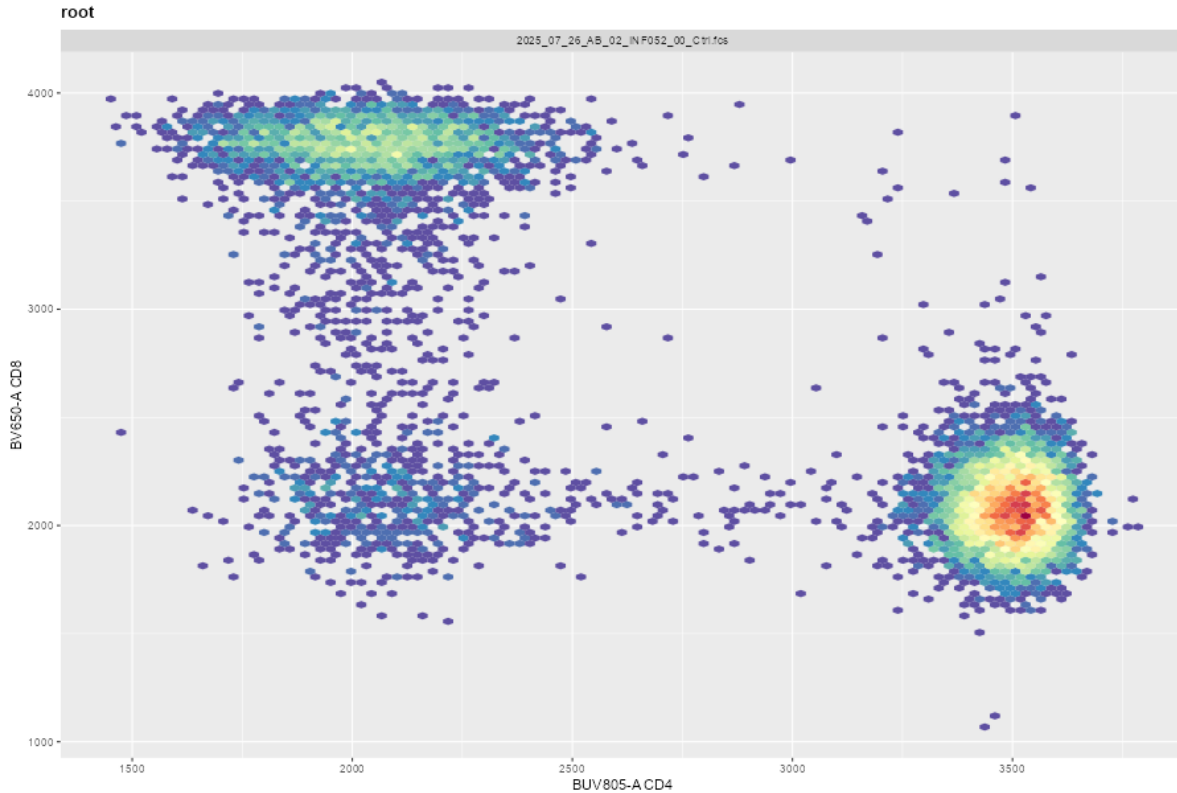
```
SFC_cytoset <- load_cytoset_from_fcs(fcs_files,
  truncate_max_range = FALSE, transformation = FALSE)
SFC_GatingSet <- GatingSet(SFC_cytoset)
```

```
SFC_Parameters <- colnames(SFC_GatingSet)
FluorophoresOnly <- SFC_Parameters[!stringr::str_detect(SFC_Parameters, "FSC|SSC|Time")]

Biexponential <- flowjo_biexp_trans(channelRange=4096, maxValue=262144,
  pos=4.5, neg=2, widthBasis=-500)
MyBiexTransform <- transformerList(FluorophoresOnly, Biexponential)
transform(SFC_GatingSet, MyBiexTransform)
```

A GatingSet with 6 samples

```
ggcyto(SFC_GatingSet[1], subset="root",  
aes(x="BUV805-A", y="BV650-A")) + geom_hex(bins=100)
```



So far, we have established a set of files (cytose), set a GatingSet and applied the necessary transformations. Now...

Manual-flowGate

David Rach usually starts with singlets

```
flowGate::gs_gate_interactive(gs=SFC_GatingSet, filterId="singlets", sample=1,  
dims=list("FSC-A", "FSC-H"), subset="root")
```

The main arguments we need to specify values for are: the name of our GatingSet (gs); provide a name for our new gate (filterId); designate which sample in the Gating Set to use for plotting

(sample=1); provide the X and Y fluorophore within a list style argument (dims=list("FSC-A", "FSC-H")); and finally the subset to be used as the parent gate. Given we haven't drawn any gates at this point, we will specify the "root" gate. Once we run the chunk (or the codes, in the console), appears the shiny app. We have to select polygon and we should draw the gate of interest. We will see the line when we click for the second time. When we have finished, then, we have to click in "done". This should be the message that we will receive:

Listening on http://127.0.0.1:6323 done! \$Gate Polygonal gate 'singlets' with 13 vertices in dimensions FSC-A and FSC-H

\$Bins [1] 256

\$Scaling [1] "unused"

To check our gates within our GatingSet:

```
plot(SFC_GatingSet)
```



The gate has been created and applied to our GatingSet. A question that may arise, was this

gate applied to just one specimen, or to all specimens in the GatingSet? Let's go ahead and check!

```
gs_pop_get_count_fast(SFC_GatingSet)
```

```
      name Population Parent Count ParentCount
      <char>      <char> <char> <int>      <int>
1: 2025_07_26_AB_02_INF052_00_Ctrl.fcs /singlets root 9824 10000 2: 2025_07_26_AB_02_INF052_00_
/singlets root 9862 10000 3: 2025_07_26_AB_02_INF100_00_Ctrl.fcs /singlets
root 9864 10000 4: 2025_07_26_AB_02_INF100_00_SEB.fcs /singlets root 9755
10000 5: 2025_07_26_AB_02_INF179_00_Ctrl.fcs /singlets root 9750 10000 6:
2025_07_26_AB_02_INF179_00_SEB.fcs /singlets root 9790 10000
```

The gate appears to have been applied across the board to all specimens, since we have counts for the respective gate for all the specimens. Let's continue, and add another gate, this time for viability dye Zombie NIR, to determine what cells are alive. We will need to modify the arguments we provide, since this will be a new gate, based on new parameters, and placed under the singlets gate.

```
flowGate::gs_gate_interactive(gs=SFC_GatingSet, filterId="live", sample=1,
dims=list("FSC-A", "Zombie NIR-A"), subset="singlets")
```

As you might notice, the included .fcs files look to have already been pre-processed, as there are few dead cells. Glancing at the y-axis, based on what we saw during Week 06 we can also deduce that behind-the-scenes the ggcyto/ggplot2 plot is not using `coords_cartesian()`, as it has zoomed in to the full extent of the data that is present. This time, we use the rectangle. When you have done, you have to click in done.

Listening on <http://127.0.0.1:6323> done! \$Gate Rectangular gate 'live' with dimensions: FSC-A: (887796.37385814,2359987.5) Zombie NIR-A: (1534.1608886719,2727.4569994262)

```
$Bins [1] 256
```

```
$Scaling [1] "unused"
```

```
plot(SFC_GatingSet)
```



And validate by checking to see if the statistics output has changed

```
gs_pop_get_count_fast(SFC_GatingSet) |>
  dplyr::filter(Population %in% "/singlets/live")
```

	name <char>	Population <char>	Parent Count <char> <int>	ParentCount <int>
1:	2025_07_26_AB_02_INF052_00_Ctrl.fcs	/singlets/live	/singlets	9421 9848
2:	2025_07_26_AB_02_INF052_00_SEB.fcs	/singlets/live	/singlets	9709 9899
3:	2025_07_26_AB_02_INF100_00_SEB.fcs	/singlets/live	/singlets	9203 9870
4:	2025_07_26_AB_02_INF100_00_SEB.fcs	/singlets/live	/singlets	9159 9778
5:	2025_07_26_AB_02_INF179_00_Ctrl.fcs	/singlets/live	/singlets	9610 9838
6:	2025_07_26_AB_02_INF179_00_SEB.fcs	/singlets/live	/singlets	9677 9841

To use the span tool, we are going to gate live T cells.

```
flowGate::gs_gate_interactive(gs=SFC_GatingSet, filterId="Tcells", sample=1,  
dims=list("Spark Blue 550-A", "Spark Blue 574-A"), subset="live")
```

Listening on http://127.0.0.1:6482 done! \$Gate Rectangular gate 'Tcells' with dimensions:
Spark Blue 550-A: (3025.2558319391,3789.8569335938)

\$Bins [1] 256

\$Scaling [1] "unused"

```
plot(SFC_GatingSet)
```



```
gs_pop_get_count_fast(SFC_GatingSet) |>  
  dplyr::filter(Population %in% "/singlets/live/Tcells")
```

	name <char>	Population <char>	Parent Count <char> <int>	ParentCount <int>
1:	2025_07_26_AB_02_INF052_00_Ctrl.fcs	/singlets/live/Tcells	/singlets/live	9416 9421
2:	2025_07_26_AB_02_INF052_00_SEB.fcs	/singlets/live/Tcells	/singlets/live	8867 9709
3:	2025_07_26_AB_02_INF100_00_Ctrl.fcs	/singlets/live/Tcells	/singlets/live	9132 9203
4:	2025_07_26_AB_02_INF100_00_SEB.fcs	/singlets/live/Tcells	/singlets/live	8509 9159
5:	2025_07_26_AB_02_INF179_00_Ctrl.fcs	/singlets/live/Tcells	/singlets/live	9330 9610
6:	2025_07_26_AB_02_INF179_00_SEB.fcs	/singlets/live/Tcells	/singlets/live	8337 9677

Having gated down to T cells, lets go ahead and add a few gates for the individual T cell populations. Let's start by drawing a polygon gate around CD4+ T cells.

```
flowGate::gs_gate_interactive(gs=SFC_GatingSet, filterId="CD4+", sample=1,
dims=list("BV650-A", "BUV805-A"), subset="Tcells")
```

Listening on http://127.0.0.1:6482 done! \$Gate Polygonal gate 'CD4+' with 15 vertices in dimensions BV650-A and BUV805-A

\$Bins [1] 256

\$Scaling [1] "unused"

Also, in case anyone was curious, flowGate retains ggcyto arguments, so if we wanted to, instead of fluorophore names, we could use alternatively use the marker names when specifying the axis for the flow cytometry plots. For example: flowGate::gs_gate_interactive(gs=SFC_GatingSet, filterId="CD4+", sample=1, dims=list("CD8", "CD4"), subset="Tcells") We can also modify the setting to overlay already existing gates when we are drawing new gates for the other two T cell subsets. This is done by providing the gate name to the overlayGates argument.

```
flowGate::gs_gate_interactive(gs=SFC_GatingSet, filterId="CD8+", sample=1,
dims=list("CD8", "CD4"), subset="Tcells", overlayGates="CD4+")
```

Listening on http://127.0.0.1:6482 done! \$Gate Polygonal gate 'CD8+' with 14 vertices in dimensions BV650-A and BUV805-A

\$Bins [1] 256

\$Scaling [1] "unused"

And we can repeat this for CD4-CD8- (Double Negative (DN)) T cells.

```
flowGate::gs_gate_interactive(gs=SFC_GatingSet, filterId="DN", sample=1,  
dims=list("CD8", "CD4"), subset="Tcells", overlayGates=c("CD4+", "CD8+"))
```

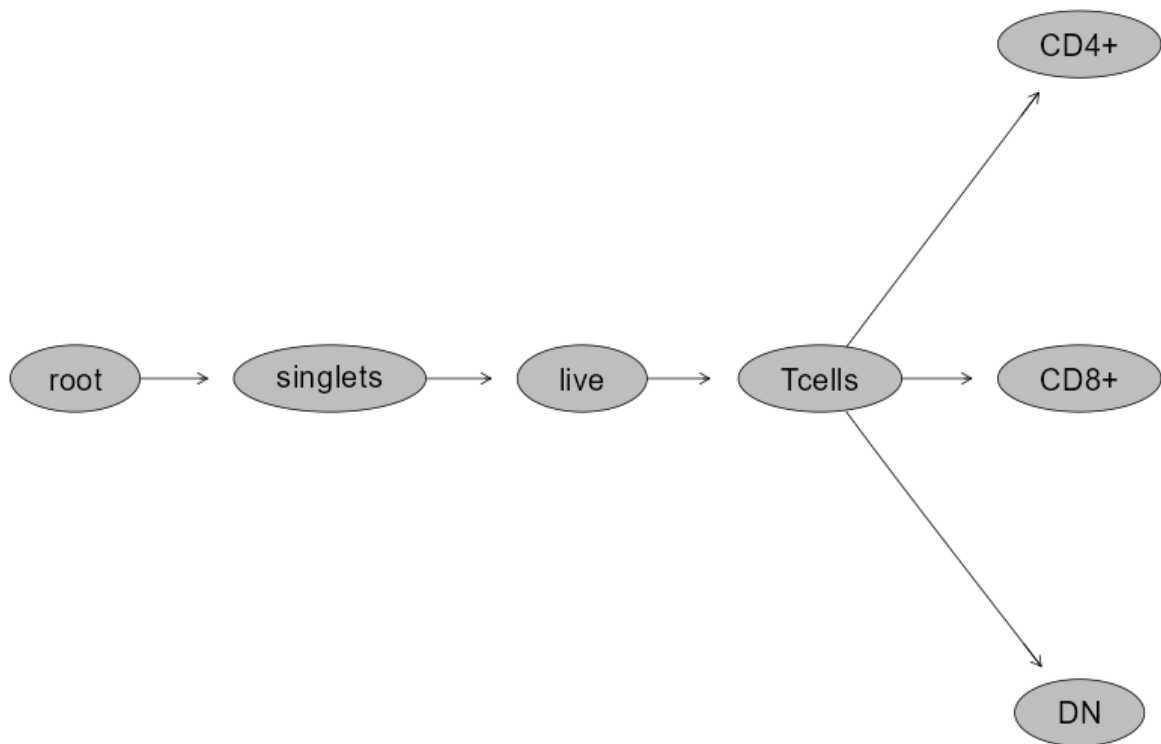
Listening on <http://127.0.0.1:6482> done! \$Gate Polygonal gate 'DN' with 14 vertices in dimensions BV650-A and BUV805-A

\$Bins [1] 256

\$Scaling [1] "unused"

And a final-check to make sure the gates for the T cell subsets were added

```
plot(SFC_GatingSet)
```



And a quick check for the statistics

```
gs_pop_get_count_fast(SFC_GatingSet) |>
  dplyr::filter(Population %in% "/singlets/live/Tcells/DN")
```

	name <char>	Population <char>	Parent Count Parent <char> <int>
1:	2025_07_26_AB_02_INF052_00_Ctrl.fcs	/singlets/live/Tcells/DN	/singlets/live/Tcells
593 9416 2:	2025_07_26_AB_02_INF052_00_SEB.fcs	/singlets/live/Tcells/DN	/singlets/live/Tcells
625 8867 3:	2025_07_26_AB_02_INF100_00_Ctrl.fcs	/singlets/live/Tcells/DN	/singlets/live/Tcells
244 9132 4:	2025_07_26_AB_02_INF100_00_SEB.fcs	/singlets/live/Tcells/DN	/singlets/live/Tcells
231 8509 5:	2025_07_26_AB_02_INF179_00_Ctrl.fcs	/singlets/live/Tcells/DN	/singlets/live/Tcells
265 9330 6:	2025_07_26_AB_02_INF179_00_SEB.fcs	/singlets/live/Tcells/DN	/singlets/live/Tcells
275 8337			

The last type of manual gate that we can implement with flowGate is a quadrant gate, in this case, let's use it to create a quadrant for our two cytokines (TNFa and IFNg). We should ideally use one of the SEB-stimulated specimens for this if we want to visualize the cytokine-expressing cells, so I have switched over to the 6th sample in the GatingSet.

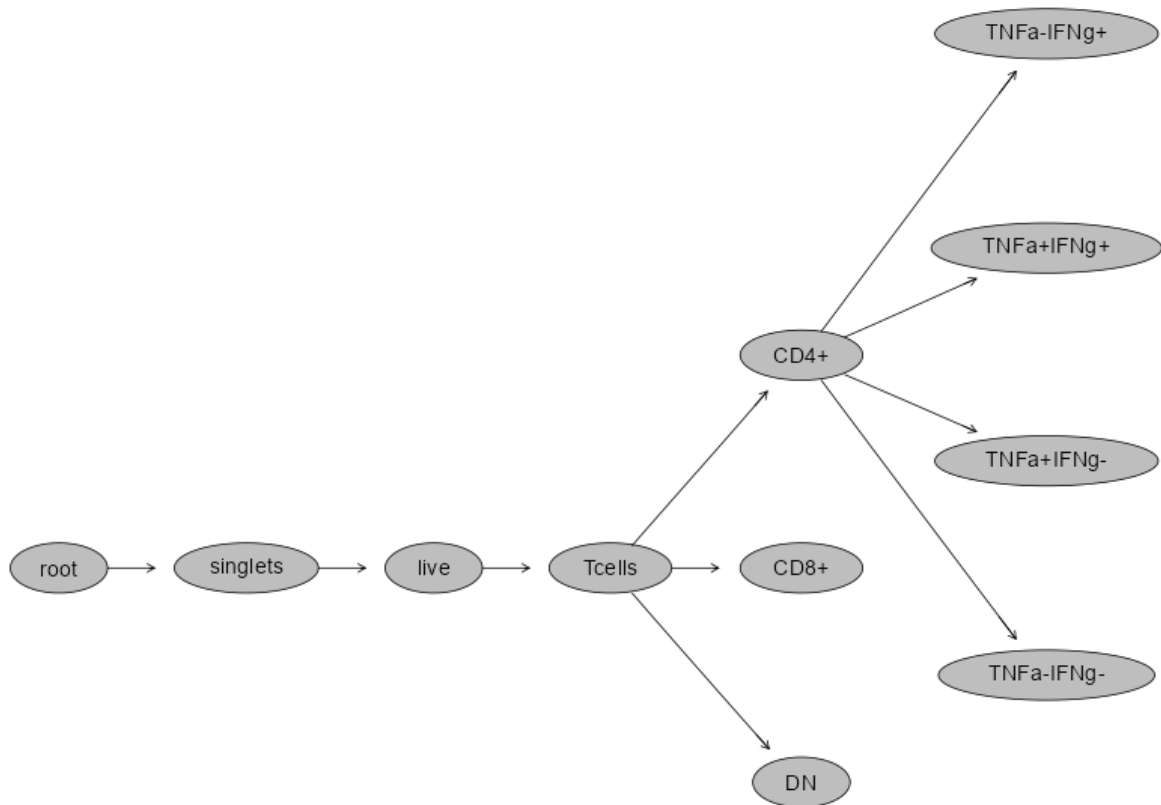
```
flowGate::gs_gate_interactive(gs=SFC_GatingSet, filterId="Cytokines", sample=6,
  dims=list("TNFa", "IFNg"), subset="CD4+")
```

Listening on http://127.0.0.1:6482 done! \$Gate Quadrant gate 'Cytokines' with dimensions:
PE-Dazzle594-A: 2833.933 BV750-A: 2898.905

\$Bins [1] 256

\$Scaling [1] "unused"

```
plot(SFC_GatingSet)
```



Redraw Gate

But what happens if we misdraw a gate and need to correct it? If you are in the process of actively drawing, you can hit the reset button and try again. But what if you had already created the gate?

```
flowGate::gs_gate_interactive(gs=SFC_GatingSet, filterId="CD161+", sample=1,
dims=list("CD161"), subset="CD8+")
```

Listening on http://127.0.0.1:6482 done! \$Gate Rectangular gate 'CD161+' with dimensions:
BV480-A: (2141.1587347718,3679.5215289787)

\$Bins [1] 256

\$Scaling [1] "unused"

Draw your gate

Reset

Done

Bins

22562,048

Gate Type:

☐ Rectangle

☐ Polygon

☒ Span

☐ Quadrant

☐ Enable Manual Coords?

X Minimum

-1000

X Maximum

50000

Y Minimum

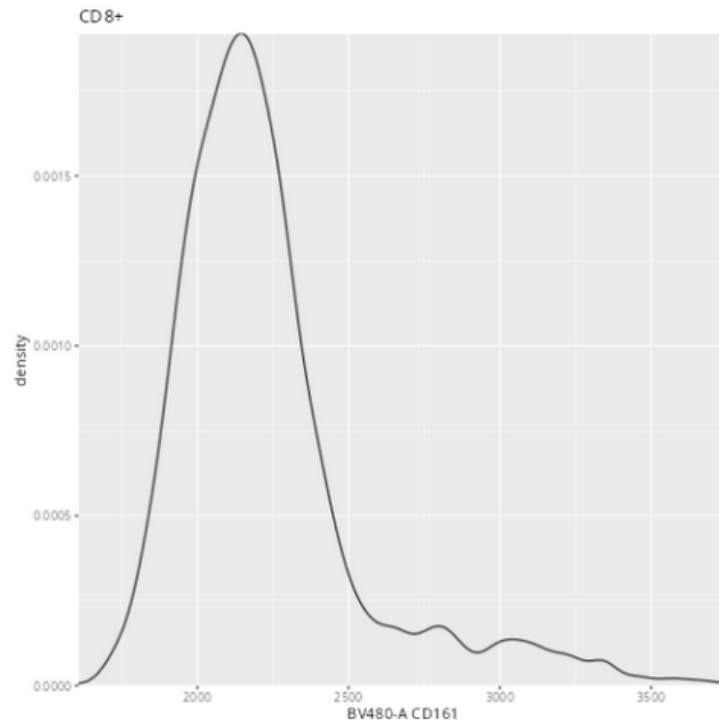
-1000

Y Maximum

50000

☐ Use FlowJo Biex?

Gate Name: CD161+
subset of: CD8+



To completely redraw an existing gate, you would need to add the “regate” argument and set it to TRUE. This will delete the existing gate, and allow you to draw a new one in its place.

```
flowGate::gs_gate_interactive(gs=SFC_GatingSet, filterId="CD161+", sample=1,  
dims=list("CD161"), subset="CD8+", regulate=TRUE)
```

Select the new gate:

Draw your gate

Reset Done

Bins

2 256 2,048

Gate Type:

☐ Rectangle

☐ Polygon

☒ Span

☐ Quadrant

☐ Enable Manual Coords?

X Minimum

-1000

X Maximum

50000

Y Minimum

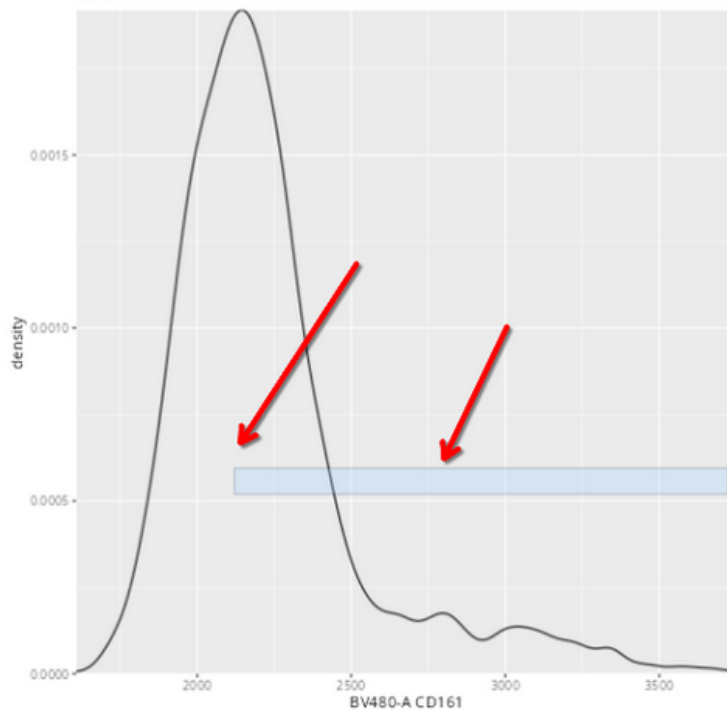
-1000

Y Maximum

50000

☐ Use FlowJo Biex?

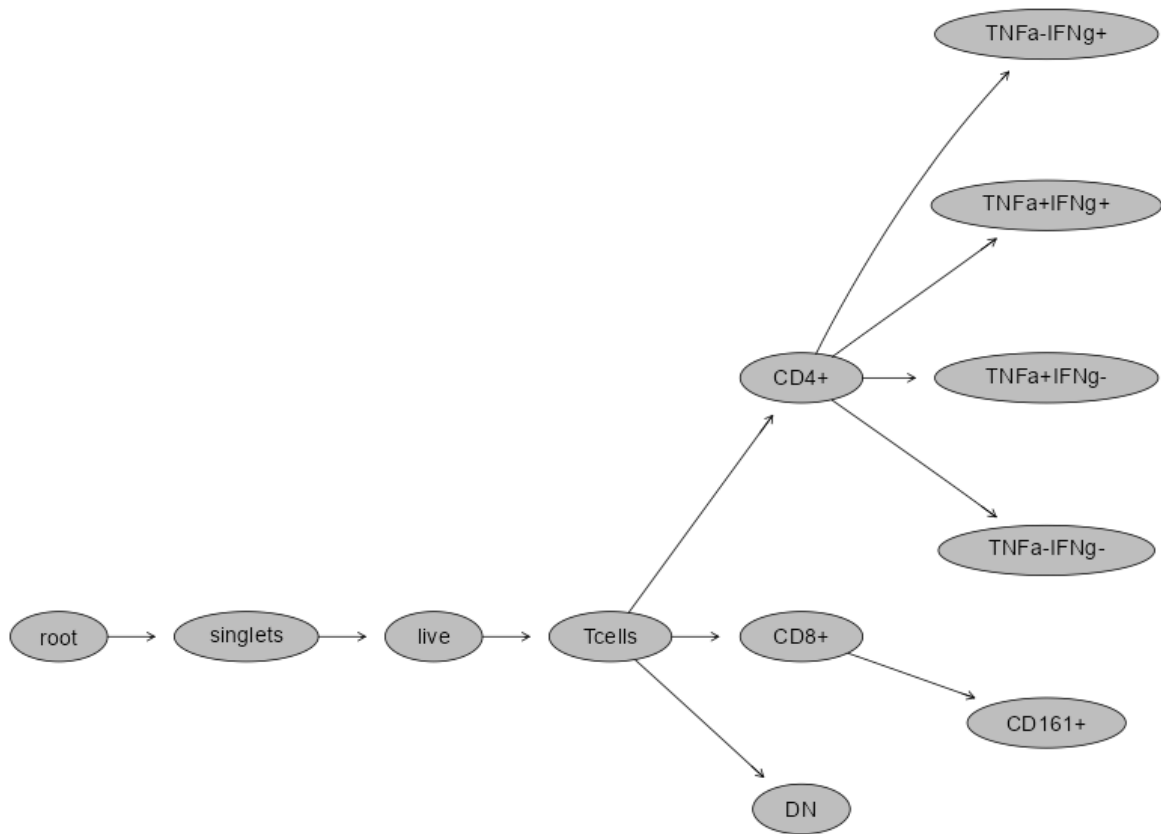
Gate Name: CD161+
subset of: CD8+
CD8+



Saving Gates

Alright, so we have put effort and drawn all our gates. But what happens if we close Positron? Well, we loose all our effort and have to redraw them again. Which doesn't sound like a particularly good use of our time.

```
plot(SFC_GatingSet)
```



Fortunately, there is a way to save our existing GatingSet object using flowWorkspace save_gs() function, allowing us to load back in the GatingSet object later on.

```
StorageLocationAndName <- file.path("C:/Users/Macarena/MacarenaLM/notes/week8-ways to gate",
save_gs(SFC_GatingSet, StorageLocationAndName)
```

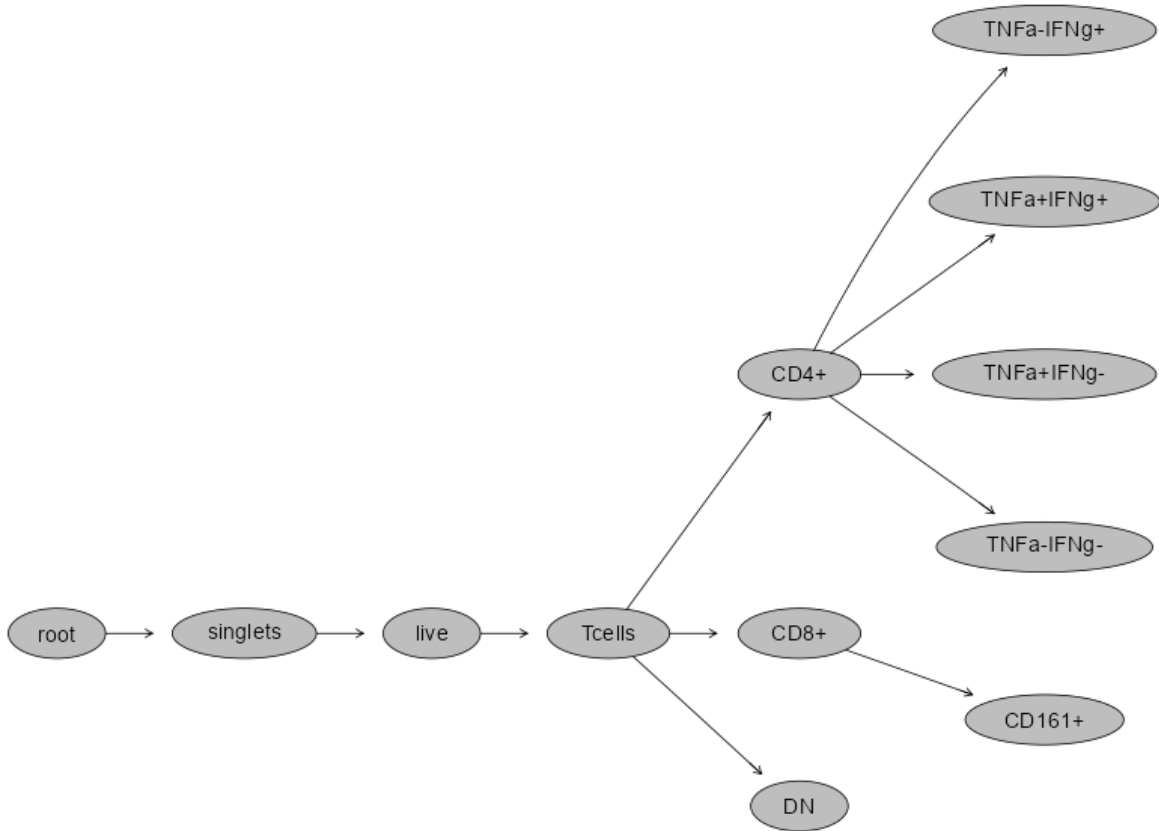
Done To reload it, use 'load_gs' function

Our GatingSet ends up being stored as its own folder with various h5 and pb objects inside. Please note, re-loading them in successfully can be file.path sensitive, so avoid messing with the folder as much as possible.

To reload back into R, we would use the load_gs() function, as the message that we receive back from R says.

```
StorageLocationAndName <- file.path("C:/Users/Macarena/MacarenaLM/notes/week8-ways to gate",
SFC_GatingSet <- load_gs(StorageLocationAndName)
```

```
plot(SFC_GatingSet)
```



```
gs_pop_get_count_fast(SFC_GatingSet) |>
  dplyr::filter(Population %in% "/singlets/live/Tcells/DN")
```

	name <char>	Population <char>	Parent Count <char>	ParentCount <int>
1:	2025_07_26_AB_02_INF052_00_Ctrl.fcs	/singlets/live/Tcells/DN	/singlets/live/Tcells	
535	9068	2:	2025_07_26_AB_02_INF052_00_SEB.fcs	/singlets/live/Tcells/DN /singlets/live/Tcells
551	6640	3:	2025_07_26_AB_02_INF100_00_Ctrl.fcs	/singlets/live/Tcells/DN /singlets/live/Tcells
230	8609	4:	2025_07_26_AB_02_INF100_00_SEB.fcs	/singlets/live/Tcells/DN /singlets/live/Tcells
206	6094	5:	2025_07_26_AB_02_INF179_00_Ctrl.fcs	/singlets/live/Tcells/DN /singlets/live/Tcells
162	8900	6:	2025_07_26_AB_02_INF179_00_SEB.fcs	/singlets/live/Tcells/DN /singlets/live/Tcells
149	5913			

Rapid Gate Deployment

In the examples above, we created separate gates by running individual chunks of code. There is an alternative method in `flowGate` to allow you to draw all your gates immediately after each other.

```
GatingTable <- tibble::tribble(
  ~filterId,    ~dims,                                ~subset,
  "singlets",   list("FSC-A", "FSC-H"),                "root",
  "live",       list("FSC-A", "Zombie NIR-A"),          "singlets",
  "Tcells",     list("CD3", "CD45"),                    "live",
  "CD4+",       list("CD8", "CD4"),                     "Tcells",
  "CD8+",       list("CD8", "CD4"),                     "Tcells",
  "DN",         list("CD8", "CD4"),                     "Tcells",
)
```

GatingTable

A tibble: 6 × 3

filterId dims subset

```
1 singlets <list [2]> root
2 live <list [2]> singlets 3 Tcells <list [2]> live
4 CD4+ <list [2]> Tcells
5 CD8+ <list [2]> Tcells
6 DN <list [2]> Tcells
```

Lets recreate our GatingSet object:

```
SFC_cytoset <- load_cytoset_from_fcs(fcs_files,
  truncate_max_range = FALSE, transformation = FALSE)
SFC_GatingSet <- GatingSet(SFC_cytoset)

SFC_Parameters <- colnames(SFC_GatingSet)
FluorophoresOnly <- SFC_Parameters[!stringr::str_detect(SFC_Parameters, "FSC|SSC|Time")]

Biexponential <- flowjo_biexp_trans(channelRange=4096, maxValue=262144,
  pos=4.5, neg=2, widthBasis=-500)
MyBiexTransform <- transformerList(FluorophoresOnly, Biexponential)
transform(SFC_GatingSet, MyBiexTransform)
```

A GatingSet with 6 samples

And now, let's apply the GatingTable scheme. As we create and click done, the next gate will automatically pop up in the Shiny App.

```
gs_apply_gating_strategy(SFC_GatingSet, gating_strategy = GatingTable)
```

Listening on http://127.0.0.1:6834 done!

Listening on http://127.0.0.1:6834 done!

Listening on http://127.0.0.1:6834 done!

Listening on http://127.0.0.1:6834 done!

Listening on http://127.0.0.1:6834 done!

Listening on http://127.0.0.1:6834 done! [[1]] [[1]]\$Gate Polygonal gate 'singlets' with 11 vertices in dimensions FSC-A and FSC-H

[[1]]\$Bins [1] 256

[[1]]\$Scaling [1] "unused"

[[2]] [[2]]\$Gate Rectangular gate 'live' with dimensions: FSC-A: (915533.625,2359987.5) Zombie NIR-A: (1535.2846546855,2655.7673484062)

[[2]]\$Bins [1] 256

[[2]]\$Scaling [1] "unused"

[[3]] [[3]]\$Gate Rectangular gate 'Tcells' with dimensions: Spark Blue 550-A: (3405.9055686318,3768.5561058662)

[[3]]\$Bins [1] 256

[[3]]\$Scaling [1] "unused"

[[4]] [[4]]\$Gate Polygonal gate 'CD4+' with 12 vertices in dimensions BV650-A and BUV805-A

[[4]]\$Bins [1] 256

[[4]]\$Scaling [1] "unused"

[[5]] [[5]]\$Gate Polygonal gate 'CD8+' with 9 vertices in dimensions BV650-A and BUV805-A

[[5]]\$Bins [1] 256

[[5]]\$Scaling [1] "unused"

[[6]] [[6]]\$Gate Polygonal gate 'DN' with 12 vertices in dimensions BV650-A and BUV805-A

```
[[6]]$Bins [1] 256
```

```
[[6]]$Scaling [1] "unused"
```

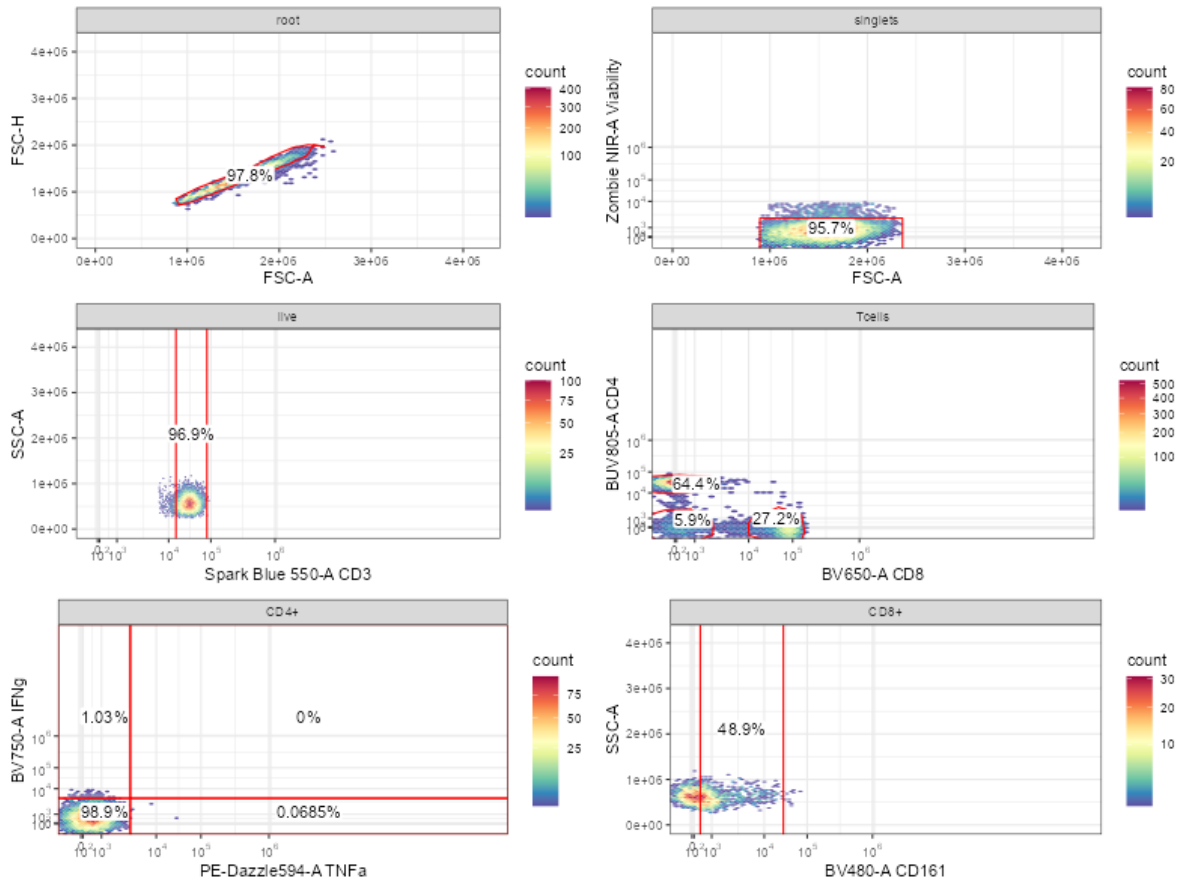
Visualizing Gates

As you may have noticed, using `flowGate`, the gates are applied generally to all specimens, and we are limited in our ability to customize to individual specimens. Consequently, being able to visualize each specimen, and regate as necessary is important if we want to have confidence in our data.

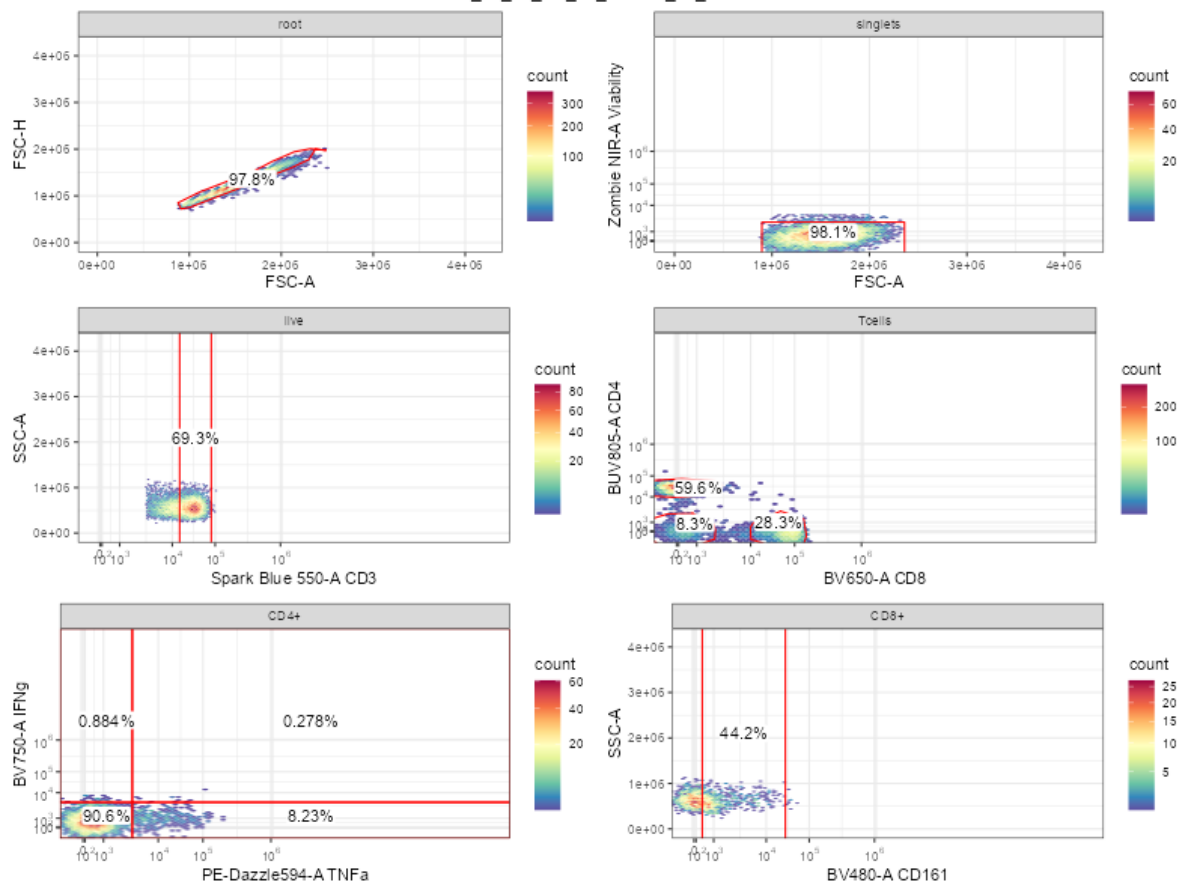
For now, the simplest way to do so is using the `autoplot()` function from the `ggcyto()` package, repeating it for every specimen in the `GatingSet`. As we go further into the course, we will learn how to iterate through the specimens, and send the plot outputs to pdf files, reducing the need to write out individual lines of code for every specimen. We will also look at how to use other visualization tools to compare individual gates across multiple specimens.

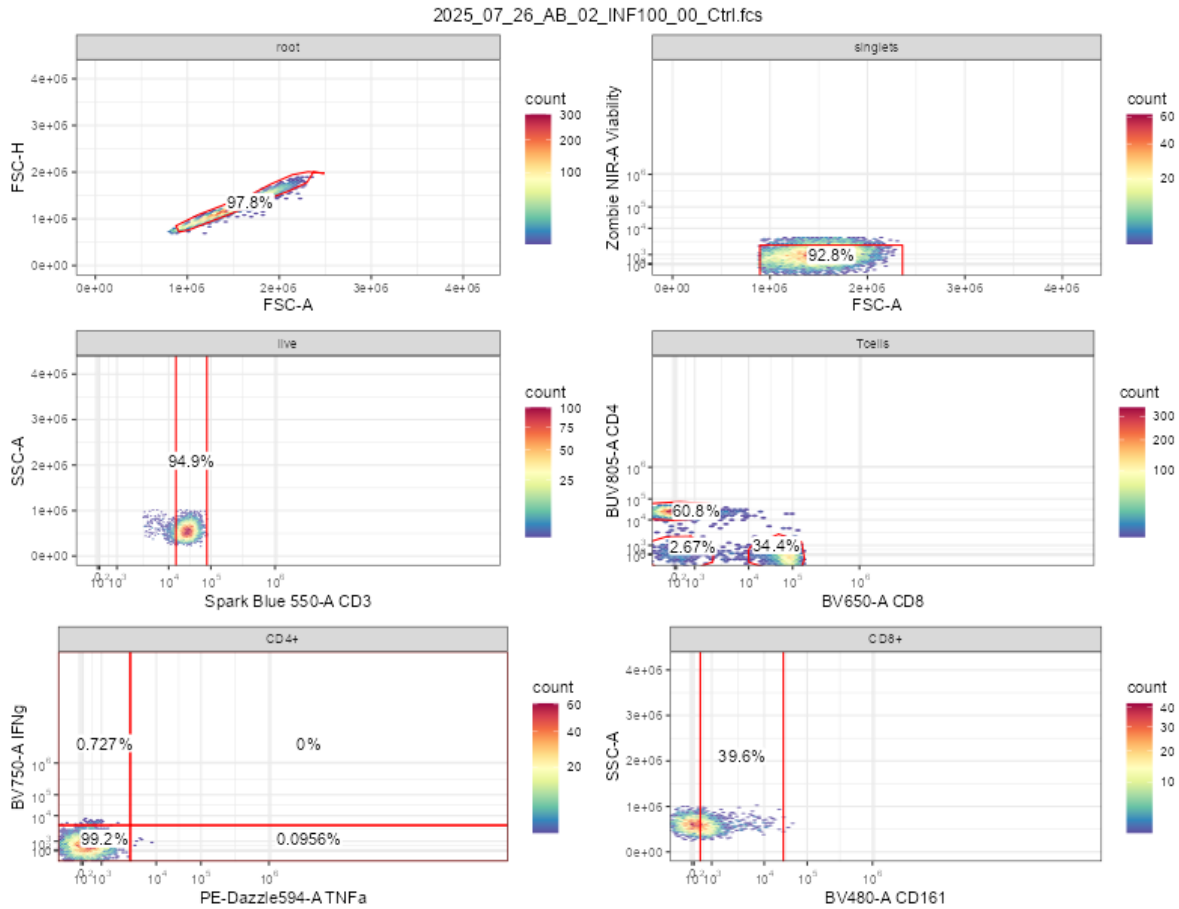
```
ggcyto::autoplot(SFC_GatingSet[[1]]) + theme_bw()
ggcyto::autoplot(SFC_GatingSet[[2]]) + theme_bw()
ggcyto::autoplot(SFC_GatingSet[[3]]) + theme_bw()
# ggcyto::autoplot(SFC_GatingSet[[4]]) + theme_bw()
# ggcyto::autoplot(SFC_GatingSet[[5]]) + theme_bw()
# ggcyto::autoplot(SFC_GatingSet[[6]]) + theme_bw()
```

2025_07_26_AB_02_INF052_00_Ctrl.fcs



2025_07_26_AB_02_INF052_00_SEB.fcs





Automated-openCyto

The other gating approach that we will be learning today is implemented by the openCyto package from Bioconductor, which is a sister package to both flowWorkspace and CytoML. It works through the use of gating template .csv files, which can be customized with various algorithmic gating functions and arguments, allowing for substantial flexibility in creating and implementing automated gating schemes.

One of the advantages of this approach (in theory), is when properly constrained, it can “better” handle some of the inter-specimen variability differences that we may encounter due to batch effects, that might otherwise throw off a fixed-manual gate applied equally across specimens. However, remember: always always always check the outputs visually.

GatingTemplate

Lets go ahead and locate an already assembled GatingTemplate for our particular dataset. This is also in the data folder, so we can separate it from our .fcs files by switching out the pattern argument

```
OurGateTemplate <- list.files(StorageLocation, pattern=".csv", full.names=TRUE)
Example <- read.csv(OurGateTemplate, check.names=FALSE)
Example
```

alias	pop	parent	dims	gating_method	gating_args	collapsed
1	singlets + root	FSC-A,FSC-H	singletGate	FALSE	NA	NA
2	lymphocytes + singlets	FSC-A, SSC-A	flowClust	K=2, target=c(2e6, 5e5)	FALSE	NA
3	live - lymphocytes	Zombie NIR-A	gate_mindensity	gate_range=c(1e4,2e4)	FALSE	NA
4	Tcells + live	Spark Blue 550-A	gate_mindensity	FALSE	NA	NA
5	CD4 + Tcells	CD4	gate_mindensity	FALSE	NA	NA
6	CD8 + Tcells	CD8	gate_mindensity	FALSE	NA	NA

Each gate is represented by an individual row. “Dims” means X and Y dimensions. The gating_method defines what kind of gate is being created, in singlets context, a singlet gating polygon. The other columns are not as easy to interpret at first glance. pop designates whether you want to keep the cells within the gate (+) or those outside the gate (-). Gating_args are specific to the gating_method, we will be using these later on when specifying gate constraints. The last four arguments are rather niche and won’t be the main focus of today, and I will refer you to the openCyto vignettes for how to use them for more specialized projects.

singletGate

For the first example, we are gating for singlets.

flowClust

Looking at the entire gatingTemplate .csv, we can see we will be implementing automated gating for 6 gates (singlets, lymphocytes, live, Tcells, CD4 and CD8). Similar to what we saw with the singlets gate, each of these gates also has corresponding parent gate and (axis) dimension information provided.

```
Example |> dplyr::select(alias, pop, parent, dims, gating_method)
```

alias	pop	parent	dims	gating_method
-------	-----	--------	------	---------------

```
1 singlets + root FSC-A,FSC-H singletGate 2 lymphocytes + singlets FSC-A, SSC-A flow-
Clust 3 live - lymphocytes Zombie NIR-A gate_mindensity 4 Tcells + live Spark Blue 550-A
gate_mindensity 5 CD4 + Tcells CD4 gate_mindensity 6 CD8 + Tcells CD8 gate_minden-
sity
```

One thing we can notice is the second gate (lymphocytes), uses the flowClust gating_method.

Example[2,]

```
alias pop parent dims gating_method gating_args collapseDataForGating groupBy preprocess-
ing_method preprocessing_args 2 lymphocytes + singlets FSC-A, SSC-A flowClust K=2,
target=c(2e6, 5e5) FALSE NA NA NA
```

One thing that is different compared to the singlets gate, is that the gating_args column has values. Behind the scenes, gating_method “flowClust” is reaching out to an R package of the same name to identify populations based on density. Especially when working with FSC-SSC (where multiple populations may be present, including lymphocytes, monocytes, granulocytes, debris, etc). having the ability to specify arguments to denote the region we are interested in can be important. In this case, target specifies the x and y coordinate we expect our population center to be roughly at. Since we are gating for lymphocytes, x is around 2e6, while the y is between 5e5 and 1e6. If we wanted a different population (ex. monocytes), we would modify the values provided accordingly to match that population.

Example[2,] |> dplyr::pull(gating_args)

```
[1] “K=2, target=c(2e6, 5e5)”
```

gate_mindensity

The last several gates are all gate_mindensity gates

Example |> dplyr::select(alias, pop, parent, dims, gating_method, gating_args)

alias	pop	parent	dims	gating_method	gating_args
1	singlets + root	FSC-A,FSC-H	singletGate		
2	lymphocytes + singlets	FSC-A, SSC-A	flowClust	K=2, target=c(2e6, 5e5)	
3	live - lym-	phocytes Zombie NIR-A	gate_mindensity	gate_range=c(1e4,2e4)	
4	Tcells + live	Spark Blue 550-A	gate_mindensity		

```
5 CD4 + Tcells CD4 gate_mindensity
```

```
6 CD8 + Tcells CD8 gate_mindensity
```

A good example of these is our CD4 gate

```
Example[5,]
```

```
alias pop parent dims gating_method gating_args collapseDataForGating groupBy preprocessing_method preprocessing_args 5 CD4 + Tcells CD4 gate_mindensity FALSE NA NA NA
```

In this case, since we are specifying pop is +, we will be keeping the positive cells (landing to the right of the gate line). Gate_mindensity is a fast calculation, finding the point with the fewest cells. It can however mistrigger, ending up with gates placed before any cell is present, or after every cell. When this occurs, we can constrain the search area by providing a gate_range to the gating_args, as we can see here for the viability gate.

```
Example[3,]
```

```
alias pop parent dims gating_method gating_args collapseDataForGating groupBy preprocessing_method preprocessing_args 3 live - lymphocytes Zombie NIR-A gate_mindensity gate_range=c(1e4,2e4) FALSE NA NA NA
```

Creating a GatingTemplate

With this broad overview of the gatingtemplate, we can now proceed with loading it in. To do so, openCyto recommends reading in the .csv file using the data.table package's fread function, to ensure the formatting is correctly retained.

```
GatesToApply <- data.table::fread(OurGateTemplate)
```

Once this is done, we can pass it to the gatingTemplate() function, which will wrap it with the required information layer needed for openCyto to add the gates to the GatingSet.

```
TheFullTemplate <- gatingTemplate(GatesToApply)
```

```
Adding population:singlets Adding population:lymphocytes Adding population:live Adding population:Tcells Adding population:CD4 Adding population:CD8
```

TheFullTemplate

— Gating Template: default with 7 populations defined

```
::: {.cell}
```

str(TheFullTemplate)

```
:::
```

```
Formal class 'gatingTemplate' [package "openCyto"] with 7 slots ..@ name : chr "default" ..@
nodes : chr [1:7] "root" "/singlets" "/singlets/lymphocytes" "/singlets/lymphocytes/live" ...
..@ edgeL :List of 7 .. ..$ root :List of 1 .. ..$ edges: int 2 .. ..$ /singlets :List of 1 .. ..$
edges: num 3 .. ..$ /singlets/lymphocytes :List of 1 .. ..$ edges: num 4 .. ..$ /singlets/lym-
phocytes/live :List of 1 .. ..$ edges: num 5 .. ..$ /singlets/lymphocytes/live/Tcells :List of
1 .. ..$ edges: num [1:2] 6 7 .. ..$ /singlets/lymphocytes/live/Tcells/CD4:List of 1 .. ..$
edges: num(0) .. ..$ /singlets/lymphocytes/live/Tcells/CD8:List of 1 .. ..$ edges: num(0)
..@ edgeData :Formal class 'attrData' [package "graph"] with 2 slots .. ..@ data :List of 6
.. .. ..$ root/singlets :List of 2 .. .. ..$ gtMethod :Formal class 'gtMethod' [package
"openCyto"] with 5 slots .. .. ..@ name : chr "singletGate" .. .. ..@
dims : chr "FSC-A,FSC-H" .. .. ..@ args : list() .. .. ..@ groupBy : chr
" " .. .. ..@ collapse: logi FALSE .. .. ..$ isReference: logi FALSE .. .. ..
..$ /singlets/singlets/lymphocytes :List of 2 .. .. ..$ gtMethod :Formal class 'gtMethod'
[package "openCyto"] with 5 slots .. .. ..@ name : chr "gate_flowclust_2d" .. ..
.. .. ..@ dims : chr "FSC-A, SSC-A" .. .. ..@ args :List of 2 .. .. ..@
.. ..$ K : num 2 .. .. ..$ target: language c(2e+06, 5e+05) .. .. ..@
groupBy : chr " " .. .. ..@ collapse: logi FALSE .. .. ..$ isReference: logi
FALSE .. .. ..$ /singlets/lymphocytes/singlets/lymphocytes/live :List of 2 .. .. ..$
gtMethod :Formal class 'gtMethod' [package "openCyto"] with 5 slots .. .. ..@ name
: chr "gate_mindensity" .. .. ..@ dims : chr "Zombie NIR-A" .. .. ..@
args :List of 1 .. .. ..$ gate_range: language c(10000, 20000) .. .. ..@
groupBy : chr " " .. .. ..@ collapse: logi FALSE .. .. ..$ isReference: logi
FALSE .. .. ..$ /singlets/lymphocytes/live/singlets/lymphocytes/live/Tcells :List of 2 .. ..
.. ..$ gtMethod :Formal class 'gtMethod' [package "openCyto"] with 5 slots .. .. ..@
..@ name : chr "gate_mindensity" .. .. ..@ dims : chr "Spark Blue 550-A" .. .. ..
.. .. ..@ args : list() .. .. ..@ groupBy : chr " " .. .. ..@ collapse: logi
FALSE .. .. ..$ isReference: logi FALSE .. .. ..$ /singlets/lymphocytes/live/Tcells/sin-
glets/lymphocytes/live/Tcells/CD4:List of 2 .. .. ..$ gtMethod :Formal class 'gtMethod'
[package "openCyto"] with 5 slots .. .. ..@ name : chr "gate_mindensity" .. .. ..
.. .. ..@ dims : chr "CD4" .. .. ..@ args : list() .. .. ..@ groupBy
: chr " " .. .. ..@ collapse: logi FALSE .. .. ..$ isReference: logi FALSE ..
.. ..$ /singlets/lymphocytes/live/Tcells/singlets/lymphocytes/live/Tcells/CD8:List of 2 ..
```

```

.. .. .. ..$ gtMethod :Formal class 'gtMethod' [package "openCyto"] with 5 slots .. .. ..
.. .. ..@ name : chr "gate_mindensity" .. .. .. ..@ dims : chr "CD8" .. .. ..
.. ..@ args : list() .. .. .. ..@ groupBy : chr "" .. .. .. ..@ collapse: logi
FALSE .. .. .. ..$ isReference: logi FALSE .. .. ..@ defaults:List of 3 .. .. ..$ gtMethod
: chr "" .. .. ..$ ppMethod : chr "" .. .. ..$ isReference: logi FALSE ..@ nodeData
:Formal class 'attrData' [package "graph"] with 2 slots .. .. ..@ data :List of 7 .. .. ..$ root
:List of 1 .. .. .. ..$ pop:Formal class 'gtPopulation' [package "openCyto"] with 3 slots ..
.. .. .. ..@ id : chr "root" .. .. .. ..@ name : chr "root" .. .. .. ..@
alias: chr "root" .. .. .. ..$ /singlets :List of 1 .. .. .. ..$ pop:Formal class 'gtPopulation'
[package "openCyto"] with 3 slots .. .. .. ..@ id : chr "/singlets" .. .. .. ..@
name : chr "+" .. .. .. ..@ alias: chr "singlets" .. .. .. ..$ /singlets/lymphocytes
:List of 1 .. .. .. ..$ pop:Formal class 'gtPopulation' [package "openCyto"] with 3 slots ..
.. .. .. ..@ id : chr "/singlets/lymphocytes" .. .. .. ..@ name : chr "+" .. ..
.. .. .. ..@ alias: chr "lymphocytes" .. .. .. ..$ /singlets/lymphocytes/live :List of 1 .. ..
.. ..$ pop:Formal class 'gtPopulation' [package "openCyto"] with 3 slots .. .. .. ..@
..@ id : chr "/singlets/lymphocytes/live" .. .. .. ..@ name : chr "-" .. .. ..
.. ..@ alias: chr "live" .. .. .. ..$ /singlets/lymphocytes/live/Tcells :List of 1 .. .. .. ..$
pop:Formal class 'gtPopulation' [package "openCyto"] with 3 slots .. .. .. ..@ id : chr
"/singlets/lymphocytes/live/Tcells" .. .. .. ..@ name : chr "+" .. .. .. ..@
alias: chr "Tcells" .. .. .. ..$ /singlets/lymphocytes/live/Tcells/CD4:List of 1 .. .. .. ..$
pop:Formal class 'gtPopulation' [package "openCyto"] with 3 slots .. .. .. ..@ id : chr
"/singlets/lymphocytes/live/Tcells/CD4" .. .. .. ..@ name : chr "+" .. .. .. ..@
..@ alias: chr "CD4" .. .. .. ..$ /singlets/lymphocytes/live/Tcells/CD8:List of 1 .. .. .. ..$
pop:Formal class 'gtPopulation' [package "openCyto"] with 3 slots .. .. .. ..@ id : chr
"/singlets/lymphocytes/live/Tcells/CD8" .. .. .. ..@ name : chr "+" .. .. .. ..@
alias: chr "CD8" .. .. .. ..@ defaults:List of 1 .. .. .. ..$ pop: chr "" ..@ renderInfo:Formal class
'renderInfo' [package "graph"] with 4 slots .. .. ..@ nodes: list() .. .. ..@ edges: list() .. .. ..@
graph: list() .. .. ..@ pars : list() ..@ graphData :List of 1 .. ..$ edgemode: chr "directed"

```

```
class(TheFullTemplate)
```

```
[1] "gatingTemplate" attr(,"package") [1] "openCyto"
```

Prepare your GatingSet

With the gatingtemplate ready to go, we need to assemble our GatingSet object. We will repeat the code that we have already been working with today above.

```

SFC_cytoset <- load_cytoset_from_fcs(fcs_files,
  truncate_max_range = FALSE, transformation = FALSE)
SFC_GatingSet <- GatingSet(SFC_cytoset)

```

```

SFC_Parameters <- colnames(SFC_GatingSet)
FluorophoresOnly <- SFC_Parameters[!stringr::str_detect(SFC_Parameters, "FSC|SSC|Time")]

Biexponential <- flowjo_biexp_trans(channelRange=4096, maxValue=262144,
  pos=4.5, neg=2, widthBasis=-500)
MyBiexTransform <- transformerList(FluorophoresOnly, Biexponential)
transform(SFC_GatingSet, MyBiexTransform)

```

A GatingSet with 6 samples

Run gt-gating

The final step is applying the template to our GatingSet.

```
gt_gating(TheFullTemplate, SFC_GatingSet)
```

Gating for 'singlets' done! done. Gating for 'lymphocytes' The prior specification has no effect when usePrior=no Using the serial version of flowClust The prior specification has no effect when usePrior=no Using the serial version of flowClust The prior specification has no effect when usePrior=no Using the serial version of flowClust The prior specification has no effect when usePrior=no Using the serial version of flowClust The prior specification has no effect when usePrior=no Using the serial version of flowClust done! done. Gating for 'live' done! done. Gating for 'Tcells' done! done. Gating for 'CD8' done! done. Gating for 'CD4' done! done. finished.

Visualizing Gates

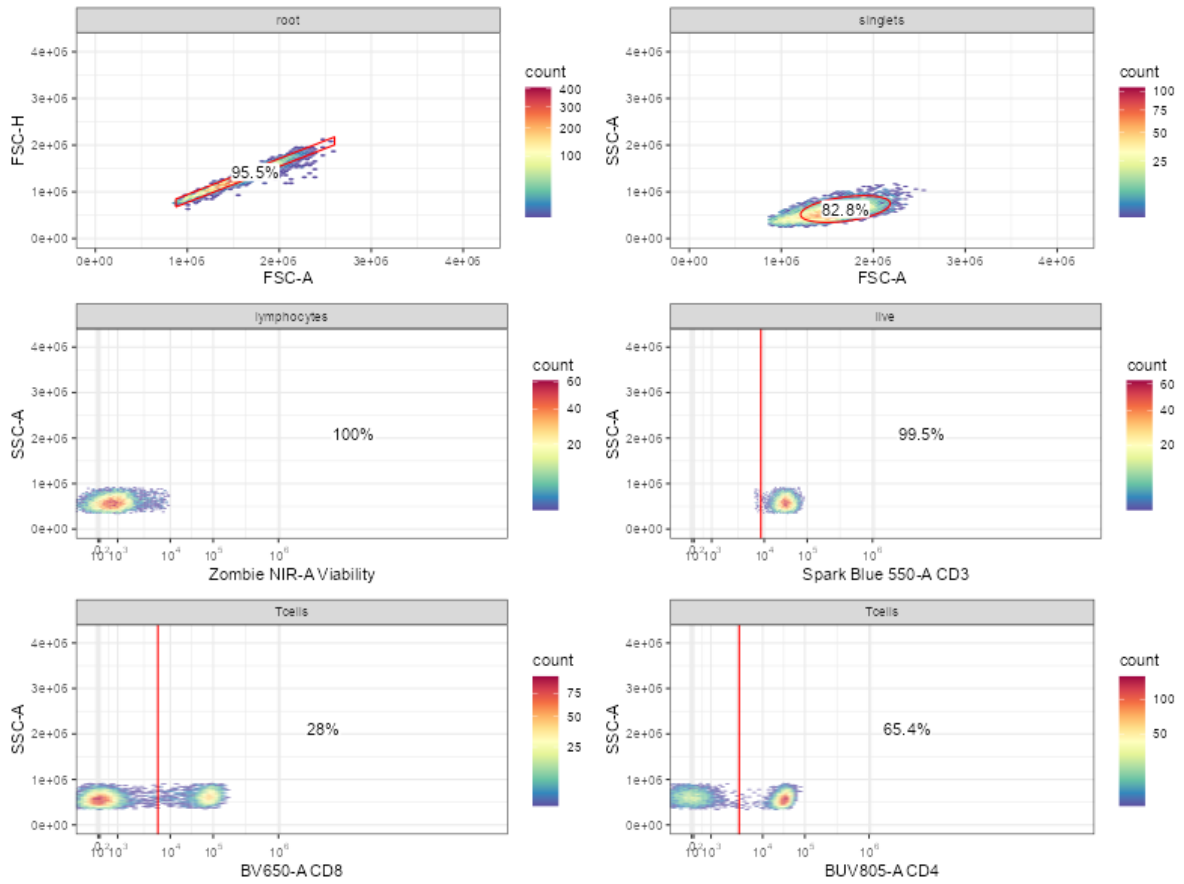
And finally, similar to the process we did for flowGate, let's go ahead and use ggcyto autoplot() function to see how we did.

```

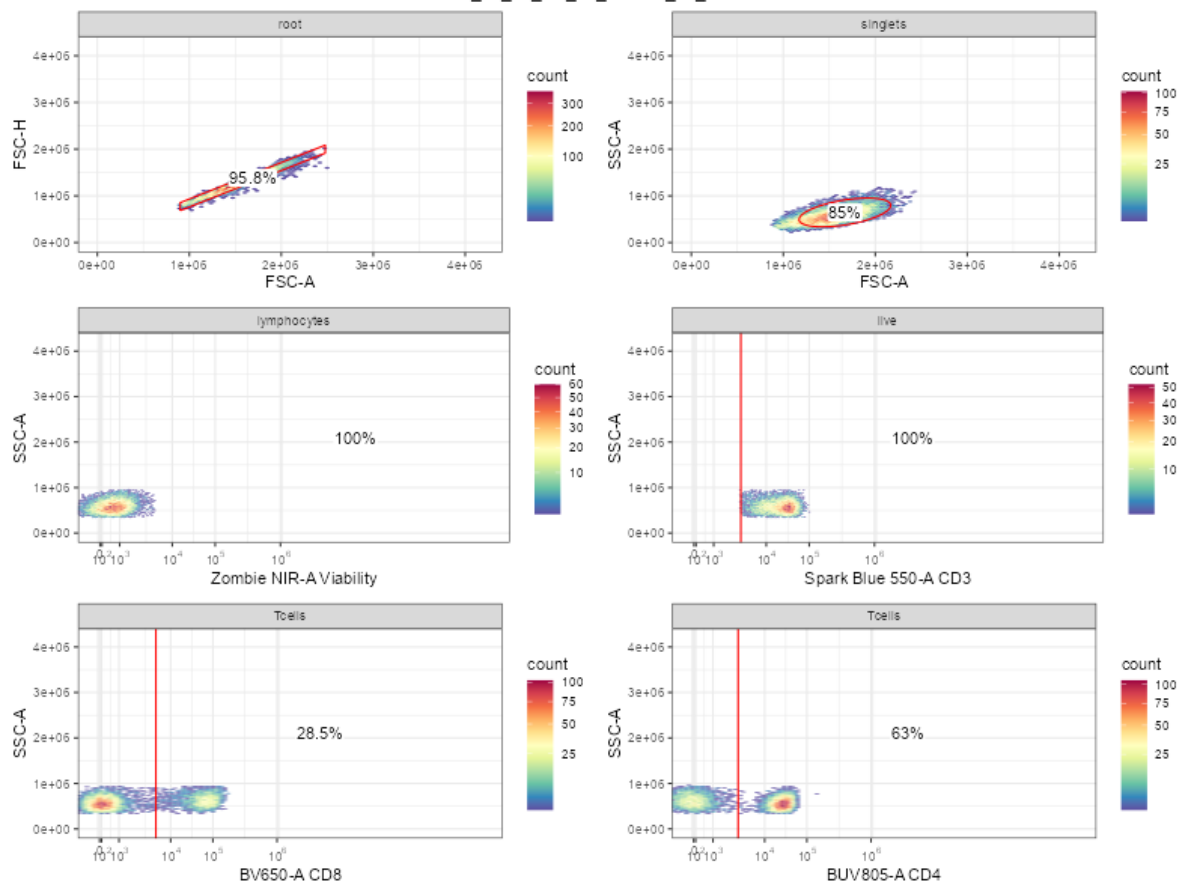
ggcyto::autoplot(SFC_GatingSet[[1]]) + theme_bw()
ggcyto::autoplot(SFC_GatingSet[[2]]) + theme_bw()
ggcyto::autoplot(SFC_GatingSet[[3]]) + theme_bw()
# ggcyto::autoplot(SFC_GatingSet[[4]]) + theme_bw()
# ggcyto::autoplot(SFC_GatingSet[[5]]) + theme_bw()
# ggcyto::autoplot(SFC_GatingSet[[6]]) + theme_bw()

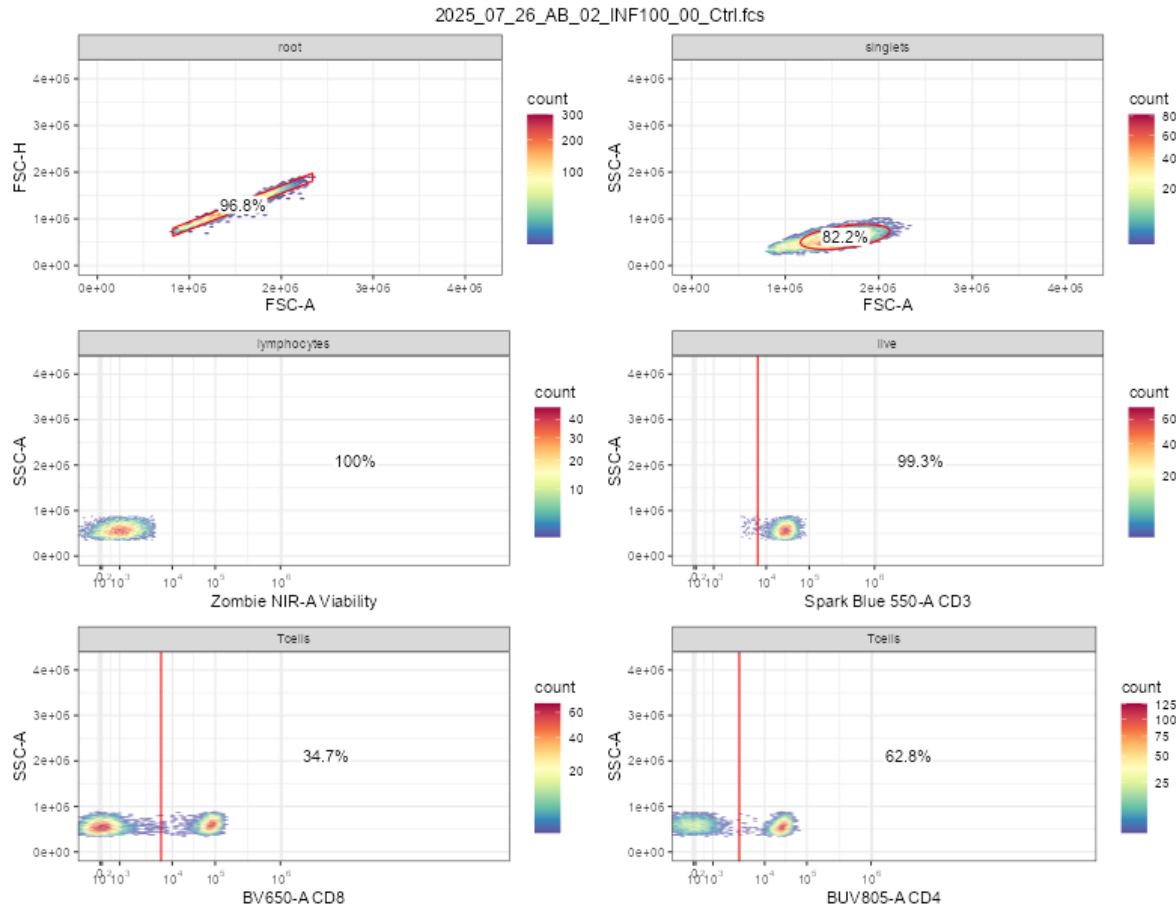
```

2025_07_26_AB_02_INF052_00_Ctrl.fcs



2025_07_26_AB_02_INF052_00_SEB.fcs





Saving for later

We can still save our openCyto-gated GatingSet using `save_gs()` (using the same steps as detailed in the `flowGate` section). However, depending on the complexity of your gating scheme, once validated and properly constrained, it is also possible to just reload everything from scratch with similar results, especially when using `gate_mindensity` (as `flowClust` is stochastic and may vary a bit).